Python 程序设计语言



第六部分 数据文件的读写操作

文件读写步骤

- 1.打开文件
- 2.处理数据
- 3.关闭文件

显示文件名为6.txt的内容，该文件只有一行

- textFile = open(“6.txt","rt") #以文本方式打开
- t=textFile.readline()
- print(t)
- textFile.close()
- binFile = open("7-1.txt","rb") #以二进制方式打开
- b=binFile.readline()
- print(b)
- binFile.close()

- 程序输出：

- 欢迎学习Python语 言
- b'\xbb\xb6\xd3\xad\xd1\xa7\xcf\xb0Python\xd3\xef\xd1\xd4'

open函数

fileobj = open(filename,mode)

fileobj是open()返回的文件对象

filename是该文件的文件名

mode是指明文件类型和操作的字符串

mode的第一个字母表明对其的操作。mode的第二个字母是文件类型：t（可省略）代表文本类型文件；b代表二进制类型文件。后面的文件处理操作均以文本方式为例。

文件打开模式

文件打开模式	含义
"r"	只读模式(默认)
"w"	覆盖写模式(不存在则新创建；存在则重写新内容)
"a"	追加模式(不存在则新创建；存在则只追加内容)
"x"	创建写模式(不存在则新创建；存在则出错)
"+"	与 r/w/a/x 一起使用，增加读写功能
"t"	文本类型
"b"	二进制类型

文件读写函数

名称	含义
open()	打开文件
read(size)	从文件读取长度为size的字符串，如果未给定或为负则读取所有内容
readline()	读取整行，返回字符串
readlines()	读取所有行并返回列表
write(s)	把字符串s的内容写入文件
writelines(s)	向文件写入一个元素为字符串的列表，如果需要换行则要自己加入每行的换行符。
seek(off, whence=0)	设置文件当前位置
tell()	返回文件读写的当前位置
close()	关闭文件。关闭后文件不能再进行读写操作。

数据的读取



- 对文件操作之前需要用 open() 函数打开文件

```
f = open('helloworld.txt', mode='r')
```

- mode 参数中的 r 指读出， w 指写入
- 打开之后将返回一个文件对象（file object），后续对文件内数据的操作都是基于这个文件对象的方法（method）来实现的

数据的读取

- 对文件数据的读取是用的 read() 方法， read() 方法将返回文件中的所有内容

```
content = f.read()
print content
f.close() #关闭文件
```

```
Hello, world!
```

- 用 print 打印所有内容会显示 Hello, world!，记得每次用完文件后，都要关闭文件 f.close()。否则，文件就会一直被Python占用，不能被其他进程使用

数据的读取

- 也可以使用 with open() as f: 在操作后自动关闭文件

```
with open('helloworld.txt') as f:
    content = f.read()
    print (content)
```

```
Hello, world!
```

数据的读取

- 在 read() 中加入数字，可指定读取的字符数

```
f = open('helloworld.txt', 'r')
content = f.read(5)
print content
f.close()
```

```
Hello
```

数据的写入

- 写入的操作和读取是类似的，不过用的是 write() 函数，同时需要将打开文件的 mode 参数设置为 w

```
f = open('helloworld_write.txt', 'w')
f.write('Hello, world!')
f.close()
```



文本文件的操作

- 常见的数据文件包括 txt 格式或者 csv（逗号分隔值文件格式）格式文

Pokemon.txt

#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary		
1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	FALSE		
2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	FALSE		
3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	FALSE		
3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	1	FALSE		
4	Charmander	Fire		309	39	52	43	60	50	65	1	FALSE		
5	Charmeleon	Fire		405	58	64	58	80	65	80	1	FALSE		
6	Charizard	Fire	Flying	534	78	84	78	109	85	100	1	FALSE		
6	CharizardMega Charizard	X	Fire	Dragon	634	78	130	111	130	85	100	1	FALSE	
6	CharizardMega Charizard	Y	Fire	Flying	634	78	104	78	159	115	100	1	FALSE	
7	Squirtle	Water		314	44	48	65	50	64	43	1	FALSE		
8	Wartortle	Water		405	59	63	80	65	80	58	1	FALSE		
9	Blastoise	Water		530	79	83	100	85	105	78	1	FALSE		
9	BlastoiseMega Blastoise	Water		630	79	103	120	135	115	78	1	FALSE		
10	Caterpie	Bug		195	45	30	35	20	20	45	1	FALSE		
11	Metapod	Bug	205	50	20	55	25	25	30	1	FALSE			
12	Butterfree	Bug	Flying	395	60	45	50	90	80	70	1	FALSE		
13	Weedle	Bug	Poison	195	40	35	30	20	20	50	1	FALSE		
14	Kakuna	Bug	Poison	205	45	25	50	25	25	35	1	FALSE		
15	Beedrill	Bug	Poison	395	65	90	40	45	80	75	1	FALSE		
15	BeedrillMega Beedrill	Bug	Poison	495	65	150	40	15	80	145	1	FALSE		
16	Pidgey	Normal	Flying	251	40	45	40	35	35	56	1	FALSE		
17	Pidgeotto	Normal	Flying	349	63	60	55	50	50	71	1	FALSE		
18	Pidgeot	Normal	Flying	479	83	80	75	70	70	101	1	FALSE		
18	PidgeotMega Pidgeot	Normal	Flying	579	83	80	80	80	135	80	121	1	FALSE	
19	Rattata	Normal		253	30	56	35	25	35	72	1	FALSE		
20	Raticate	Normal		413	55	81	60	50	70	97	1	FALSE		
21	Spearow	Normal	Flying	262	40	60	30	31	31	70	1	FALSE		
22	Fearow	Normal	Flying	442	65	90	65	61	61	100	1	FALSE		
23	Ekans	Poison		288	35	60	44	40	54	55	1	FALSE		

文本文件的读取

- 通过 open() 函数打开文件，返回文件对象
- 对文件对象进行读取操作，除了前面介绍的 read() 之外还有两种读取数据的方法： readline() 是每次读入一条数据的方式， readlines() 是一次性读入文件所有数据

```
path = 'Pokemon.txt'
f = open(path, 'r') #打开文件并存储到f变量中
content = f.readlines() #一次性读入所有数据
f.close()
print content[0] #打印content这个列表中的第一个元素
print content[1]
```

```
#   Name    Type 1  Type 2  Total   HP  Attack  Defense Sp. Atk Sp. Def Speed   Generation  Legendary

1   Bulbasaur   Grass   Poison  318 45  49  49  65  65  45  1   FALSE
```

文本文件的读取

- readlines() 读取后得到的是每行数据组成的列表，但是一行样本数据全部存储为一个字符串，并且数据读入后并没有将换行符去掉（windows系统的换行符是 \r\n，linux系统的换行符号是 \n）
- 在读入数据之后，用 for 循环对每一个元素去除换行符，并将每一个变量值用字符串处理方法 .split() 分隔开来

```
#   Name    Type 1  Type 2  Total   HP  Attack  Defense Sp. Atk Sp. Def Speed   Generation  Legendary
```

文本文件的读取

- .strip() 本身是一个对字符串指定字符去除的方法，但括号里参数为空的时候，就会去除 \r \n \t

```
content_new = [] #新建一个列表存储进行处理后的数据
for con in content:
    tmp = con.strip()  #去除换行符
    tmp = tmp.split('\t')  #以'\t' 的方式对字符串进行分隔，并返回一个列表
    content_new.append(tmp)  #将处理后的数据添加到content_new中

print content_new[0]
print content_new[1]
```

```
['#', 'Name', 'Type 1', 'Type 2', 'Total', 'HP', 'Attack', 'Defense', 'Sp. Atk', 'Sp. Def', 'Speed',
['1', 'Bulbasaur', 'Grass', 'Poison', '318', '45', '49', '49', '65', '65', '45', '1', 'FALSE']
```

文本文件的写入

- 设置参数 mode=w。 write() 、 writelines() 是两个对文件对象的写入数据的方法。write() 是逐次写入， writelines() 可对一个列表里的所有数据一次性写入文件中
- 如果有换行需要，则要在每条数据后增加换行符，同时用字符串 .join() 的方法将

Pokemon_write.txt

#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	FALSE

```
path = 'Pokemon_write.txt'
f = open(path, 'w')

#写入Pokemon前两行数据
content = [['#', 'Name', 'Type 1', 'Type 2', 'Total', 'HP', 'Attack', 'Defense', 'Sp. Atk', 'Sp. Def', 'Speed', 'Generation', 'Legendary'],
           ['1', 'Bulbasaur', 'Grass', 'Poison', '318', '45', '49', '49', '65', '65', '45', '1', 'FALSE']]

#循环content中内容
for con in content:
    con = '\t'.join(con)  #是用.join()将列表中每个数值用'\t'联合组成一个字符串
    con = con + '\r\n' #每行后添加换行符，如果是linux系统则换成'\n'
    f.write(con)
f.close()
```

逗号分隔值文件的操作

- csv文件用逗号，作为分隔符

```
#,Name,Type 1,Type 2,Total,HP,Attack,Defense,Sp. Atk,Sp. Def,Speed,Generation,Legendary
1,Bulbasaur,Grass,Poison,318,45,49,49,65,65,45,1,False
2,Ivysaur,Grass,Poison,405,60,62,63,80,80,60,1,False
3,Venusaur,Grass,Poison,525,80,82,83,100,100,80,1,False
3,VenusaurMega Venusaur,Grass,Poison,625,80,100,123,122,120,80,1,False
4,Charmander,Fire,,309,39,52,43,60,50,65,1,False
5,Charmeleon,Fire,,405,58,64,58,80,65,80,1,False
6,Charizard,Fire,Flying,534,78,84,78,109,85,100,1,False
6,CharizardMega Charizard X,Fire,Dragon,634,78,130,111,130,85,100,1,False
6,CharizardMega Charizard Y,Fire,Flying,634,78,104,78,159,115,100,1,False
7,Squirtle,Water,,314,44,48,65,50,64,43,1,False
8,Wartortle,Water,,405,59,63,80,65,80,58,1,False
9,Blastoise,Water,,530,79,83,100,85,105,78,1,False
9,BlastoiseMega Blastoise,Water,,630,79,103,120,135,115,78,1,False
10,Caterpie,Bug,,195,45,30,35,20,20,45,1,False
```

逗号分隔值文件的读取

- 利用Python内置的 csv 模块读取数据

```
import csv #引入csv模块

path = 'Pokemon.csv'
f = open(path, 'r')

reader = csv.reader(f) #用csv.reader()方法生成reader对象
content = []  #用于数据存储

#for循环，从reader中逐个添加存储
for con in reader:
    content.append(con)
f.close()

print content[0]
print content[1]
```

```
['#', 'Name', 'Type 1', 'Type 2', 'Total', 'HP', 'Attack', 'Defense', 'Sp. Atk', 'Sp. Def', 'Speed', 'Ge
['1', 'Bulbasaur', 'Grass', 'Poison', '318', '45', '49', '49', '65', '65', '45', '1', 'False']
```

逗号分隔值文件的写入

- 引入 csv 模块 open() 打开文件，使用 csv.writer() 作为写入器， writerow() 方法逐行写入

```
import csv

path = 'Pokemon_write.csv'

#写入Pokemon前两行数据
content = [['#', 'Name', 'Type 1', 'Type 2', 'Total', 'HP', 'A
           ['1', 'Bulbasaur', 'Grass', 'Poison', '318', '45',

f = open(path, 'w')
writer = csv.writer(f) #生成writer对象对存储

#for循环，逐个对内容写入文件中
for con in content:
    writer.writerow(con)

f.close()
```

Pokemon_write.csv

```
#,Name,Type 1,Type 2,Total,HP,Attack,Defense,Sp. Atk,Sp. Def,Speed,Generation,Legendary
1,Bulbasaur,Grass,Poison,318,45,49,49,65,65,45,1,FALSE
```

例 文件复制

- cj.txt 文件是学生一门课的成绩，它的内容如下：
- 97 80 93 69 87 90 84 94 75 76 89 83 83 33 72 48 66 86 98
- 89 89 88 87 63 87 81 100 80 37 68 71 77 98 66 47 29 87 93
- 96 100 70 85 83 35
- 需要把这个文件的内容复制到“cjback.txt”文件中。复制文件不需要考虑行结构，用read函数就可以了。
- 程序：
- source=open("cj.txt","r")
- back=open("cjback.txt","w")
- s=source.read()
- back.write(s)
- source.close()
- back.close()

例 计算总评分

- 文件score.txt是学生一学期的成绩，每一行代表一个学生的成绩，由笔试、平时和实验三部分构成。总评=笔试*50%+平时*25%+实验*25%

学号	姓名	专业	笔试	平时	实验
2050921018	詹延峰	计算数学	65	85	76
2050921036	李小鹏	金融学类	86	95	85
2050921039	裴凡法	经济学类	86	95	65
2040912116	茅舒瑶	社会保障	90	95	100
2050912017	陈见影	化学工程	62	75	92
2050912064	梅钦钦	材料科学	87	95	80
2050109153	王影平	大气科学	86	89	72
2050151003	韩平医	化学工程	82	99	60

readlines函数

- f=open("score.txt","r")
- s=f.readlines()
- print(s)

- 运行结果：

['学号 姓名 专业 笔试 平时 实验\n', '2050921018 詹延峰 计算数学 65 85 76\n', '2050921036 李 小鹏 金融学类 86 95 85\n', '2050921039 裴凡法 经济学类 86 95 65\n', '2040912116 茅舒瑶 社会保障 90 95 100\n', '2050912017 陈 见影 化学 工程 62 75 92\n', '2050912064 梅 钦钦 材料科学 87 95 80\n', '2050109153 王影平 大 气科学 86 89 72\n', '2050151003 韩平医 化学 工程 82 99 60\n']

文件逐行读取

- f=open("score.txt","r")
- for line in f.readlines():
- 　　print(line)

- 程序输出：
-
- 学号 姓名 专业 笔试 平时 实验
- 2050921018 詹延峰 计算数学 65 85 76

- 2050921036 李 小鹏 金融学类 86 95 85

- 2050921039 裴凡法 经济学类 86 95 65
- 。。。。。。

计算总评分

- f=open("score.txt","r")
- head=f.readline() #读表头行
- newhead=head[:9]+' '+head[9:18]+' '+head[18:-1]+' 总评成绩' #加空格对齐
- print(newhead)
- for line in f.readlines():
- 	l=line.split()
- 	#求总评分
- 	s=round(int(l[3])*0.5+int(l[4])*0.25+int(l[5])*0.25,2)
- 	l[4]=' '+l[4] #加空格对齐
- 	l[5]=' '+l[5] #加空格对齐
- 	print(' '.join(l)+' '+str(s)) #加空格对齐

总评成绩

学号	姓名	专业	笔试	平时	实验	总评成绩
2050921018	詹延峰	计算数学	65	85	76	72.75
2050921036	李小鹏	金融学类	86	95	85	88.0
2050921039	裴凡法	经济学类	86	95	65	83.0
2040912116	茅舒瑶	社会保障	90	95	100	93.75
2050912017	陈见影	化学工程	62	75	92	72.75
2050912064	梅钦钦	材料科学	87	95	80	87.25
2050109153	王影平	大气科学	86	89	72	83.25
2050151003	韩平医	化学工程	82	99	60	80.75

重定向

- sys.stdin　标准输入
- sys.stdout标准输出
- sys.stderr 标准错误输出

- import sys
- s=sys.stdin.readlines() #从文件读入变为从键盘输入
- print(s)

词频统计

- 请编写程序，对一段英文文本，统计其中所有不同单词的个数，以及词频最大的10%的单词。所谓“单词”，是指由不超过80个单词字符组成的连续字符串，但长度超过15的单词将只截取保留前15个单词字符。而合法的“单词字符”为大小写字母、数字和下划线，其它字符均认为是单词分隔符。注意“单词”不区分英文大小写，例如“PAT”和“pat”被认为是同一个单词。输入给出一段非空文本，最后以符号#结尾。输入保证存在至少10个不同的单词。输出按照词频递减的顺序，按照“词频:单词”的格式输出词频最大的前10%的单词。若有并列，则按递增字典序输出。

输入处理

- s=sys.stdin.read() 表示重定向为键盘输入
- s[:s.find('#')] 表示取输入字符串，以符号"#"结尾

产生词频字典

```
# k: 异常字符
# i.isalnum：判断字符变量i是否为字母或数字，若是则返回非零，否则返回零。
for k in set([i for i in strs if i.isalnum()==False and i!='_']):
   strs=strs.replace(k, ' ') #把strs中的异常字符替换为空格，作为单词分隔符
strs=strs.rstrip(' ').lower().split() #全部变小写
counts=dict()
for i in strs:
      k=i[:15]
      if k not in counts:
            counts[k]=1
      else:
            counts[k]+=1
```

排序及输出

- #词频递减的顺序，若有并列，则按递增字典序
- ans=sorted(counts.items(), key=lambda x:(-x[1], x[0]))
- print(len(counts))
-
- for i in range(0,int(0.1*len(counts))): 词频最大的前10%的单词
- 　　print(str(ans[i][1])+':'+ans[i][0])

完整程序

```
import sys
s=sys.stdin.read()
strs=s[:s.find('#')]        #"以符号#结尾"
for k in set([i for i in strs if i.isalnum()==False and i!='_']):
    strs=strs.replace(k, ' ') #其它字符均认为是单词分隔符
strs=strs.rstrip(' ').lower().split() #全部变小写
counts=dict()
for i in strs:
        k=i[:15]
        if k not in counts:
                counts[k]=1
        else:
                counts[k]+=1
#词频递减的顺序，若有并列，则按递增字典序
ans=sorted(counts.items(), key=lambda x:(-x[1], x[0]))
print(len(counts))

for i in range(0,int(0.1*len(counts))): 词频最大的前10%的单词
        print(str(ans[i][1])+':'+ans[i][0])
```

用pandas读写各种类型文件

读取函数	写入函数	含义
read_csv	to_csv	读写csv 文件
read_excel	to_excel	读写excel 文件
read_json	to_json	读写json 文件
read_clipboard	to_clipboard	读写粘贴板内容

读取CSV和表格文件

- import plotly as py
- from plotly import figure_factory as FF
- import plotly.graph_objs as pygo
- import pandas as pd
- pyplt = py.offline.plot　　#离线模式
- data = pd.read_csv("score.csv",encoding="GBK")
- table = FF.create_table(data)　　#产生表格
- pyplt(table, show_link=False)

- "GB2312"、"GBK"和"CP936"都是用两个字节表示中文。"GB2312"是中国制定的中文编码，"GBK"是"GB2312"的扩展，"CP936"是在"GB2312"基础上开发的汉字编码。

写网页文件

- 产生网页文件'scoregp.html

```
import plotly as py
from plotly import figure_factory as FF
import plotly.graph_objs as pygo
import pandas as pd
pyplt = py.offline.plot #离线模式
data = pd.read_csv("scoregp.csv",encoding="GBK")

table = FF.create_table(data)  #产生表格

pyplt(table, filename='scoregp.html',show_link=False)
```

读写Excel 文件

- 要装xlrd模块
- 读score.xlsx文件
- 写入scoregp.xlsx文件

- import pandas as pd
- data = pd.read_excel("score.xlsx")
- data["总评"]=data["笔试"]*0.5+data["平时"]*0.25+data["实验"]*0.25 data.to_excel("scoregp.xlsx",index=0)

JSON 文件读写

- JSON建构于两种结构：

- “名称/值”对的集合（A collection of name/value pairs）。不同的语言中，它被理解为对象 （object），纪录（record），结构（struct），字典（dictionary），哈希表（hash table），有键列表（keyed list），或者关联数组 （associative array）。Python中对应字典类型。
- 值的有序列表（An ordered list of values）。在大部分语言中，它被理解为数组（array）。这些都是常见的数据结构。事实上大部分现代计算机语言都以某种形式支持它们，这使得一种数据格式在同样基于这些结构的编程语言之间交换成为可能。Python中对应列表类型。

to_json和read_json

- 读取Excel 文件"score.xlsx”，用to_json函数产生JSON 文件data.json, 最后用read_json读JSON 文件“data.json”

- import pandas as pd
- data = pd.read_excel("score.xlsx")
- data.to_json("data.json",force_ascii=False)
- jsondata=pd.read_json("data.json")
- print(jsondata)

总结

- 文本文件读写操作、逗号分隔值文件读写操作
- open()、write()、readlines() 、 .strip()、 .join()
- csv模块、 csv.reader()、 csv.writer()



第七部分 编写函数处理数据

函数的语法规则
参数设定
lambda函数

函数的语法规则

```
def function_name(parameter1, parameter2, ...):
    expression ...
    return expression
```

- 实例：平方函数，输入变量 x 返回其平方值 x^2

```
def square(x):
    s = x*x
    return s
```

函数的语法规则

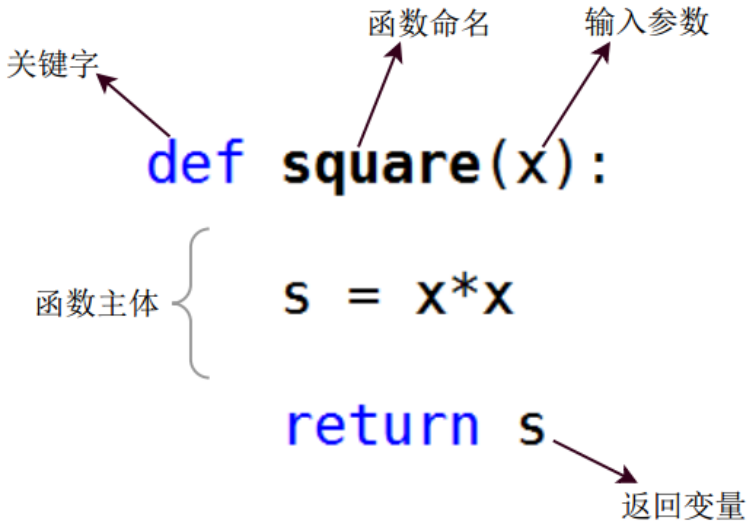
- 内置函数
- >>>len(“hello”)
- 5
- 函数是重用的程序段。它们允许你给一块语句命名一个名字，这是函数定义。然后你可以在你的程序的任何地方使用这个名称运行这个语句块，这被称为调用函数

- 自定义函数
- def 函数名(参数表):
- 　　函数体

- 如定义函数：$y=x^2+1$
- def f(x):
- 　value=x**2+1
- 　return value

- n=int(input())
- y=f(n)
- print(y)

函数的语法规则




```
a = 3

#调用函数，并打印出来
print square(a)

#调用函数，保存至变量b
b = square(a)
```

```
9
```

参数设定

- 函数取得的参数值是你提供给函数的值，这样函数就可以利用这些值做一些事情。这些参数就像变量一样，只不过它们的值是在我们调用函数的时候定义的，而非在函数本身内赋值。参数在函数定义的圆括号内指定，用逗号分割。当我们调用函数的时候，我们以同样的方式提供值。函数中的参数名称为形参而你提供给函数调用的值称为实参。

- 实参(argument)：
- 全称为"实际参数"是在调用时传递给函数的参数. 实参可以是常量、变量、表达式、函数等，无论实参是何种类型的量，在进行函数调用时，它们都必须具有确定的值， 以便把这些值传送给形参。 因此应预先用赋值，输入等办法使实参获得确定值。
-
- 形参(parameter)：
- 全称为"形式参数" 由于它不是实际存在变量，所以又称虚拟变量。是在定义函数名和函数体的时候使用的参数,目的是用来接收调用该函数时传入的参数.在调用函数时，实参将赋值给形参。因而，必须注意实参的个数，类型应与形参一一对应，并且实参必须要有确定的值。

参数设定

- 实参(argument)：

全称为"实际参数"是在调用时传递给函数的参数. 实参可以是常量、变量、表达式、函数等， 无论实参是何种类型的量，在进行函数调用时，它们都必须具有确定的值， 以便把这些值传送给形参。 因此应预先用赋值，输入等办法使实参获得确定值。

- 形参(parameter)：

全称为"形式参数" 由于它不是实际存在变量，所以又称虚拟变量。是在定义函数名和函数体的时候使用的参数,目的是用来接收调用该函数时传入的参数.在调用函数时，实参将赋值给形参。因而，必须注意实参的个数，类型应与形参一一对应，并且实参必须要有确定的值。

参数设定

- 几种参数设定方法
  - 顺序传入
  - 关键词
  - 默认参数
  - 不定长参数

顺序传入

- 按照输入参数列表的顺序传入
- x=1, y=2, z=3

```
#  多项式 s = 1 + 2*x + y^2 + z*y
def polynomial(x, y, z):
    s = 1 + 2*x + y*y + z*y
    return s

#  按位置输入
print polynomial(1, 2, 3)
```

```
13
```

顺序传入

- Python处理参数的方式要比其他语言更加灵活。其中，最熟悉的参数类型是位置参数，传入参数的值是按照顺序依次复制过去的。
- from math import sqrt
- def dis(x1,y1,x2,y2): #求平面上两点距离
- 　　print("x1={},y1={},x2={},y2={}".format(x1,y1,x2,y2))
- 　　return sqrt((x1-x2)**2+(y1-y2)**2)
- print(dis(1,3,4,5))

关键词

- 为了避免位置参数带来的混乱，调用参数时可以指定对应参数的名字，这是关键字参数，它甚至可以采用与函数定义不同的顺序调用.
- from math import sqrt

- def dis(x1,y1,x2,y2): #求平 面上两点距离
- 	print("x1={},y1={},x2={},y2={}".format(x1,y1,x2,y2))
- 	return sqrt((x1-x2)**2+(y1-y2)**2)
- print(dis(x1=1,y2=5,y1=3,x2=4))

关键词

- 直接在参数列表里设定关键词确定

```
#  多项式 s = 1 + 2*x + y^2 + z*y
def polynomial(x, y, z):
    s = 1 + 2*x + y*y + z*y
    return s
```

```
print polynomial(x=1, y=2, z=3)
```

```
13
```

关键词

- 可以将位置和关键词的方法混合使用

```
print polynomial(1, y=2, z=3)
```

```
13
```

- 如果传入的第一个参数是用关键词传入的，那么后面每个参数都需要是关键词传入，否则会出现语法错误

```
print polynomial(x=1, 2, z=3)
```

```
  File "<ipython-input-11-86ff7142f0bc>", line 1
    print (polynomial(x=1, 2, z=3))
                           ^
SyntaxError: positional argument follows keyword argument
```

关键词

- 如果同时出现两种参数形式， 首先应该写的是位置参数，然后是关键字参数
- from math import sqrt
- def dis(x1,y1,x2,y2): #求平 面上两点距离
- print("x1={},y1={},x2={},y2={}".format(x1,y1,x2,y2))
- return sqrt((x1-x2)**2+(y1-y2)**2)
- print(dis(1,3,y2=5,x2=4))
- 下面出错
- from math import sqrt
- def dis(x1,y1,x2,y2): #求平面上两点距离
- print("x1={},y1={},x2={},y2={}".format(x1,y1,x2,y2))
- return sqrt((x1-x2)**2+(y1-y2)**2)
- print(dis(1,y1=3,4,5))

默认参数

- 当调用方没有提供对应的参数值时，你可以指定默认参数值。如果你提供参数值，在调用时会代替默认值
- from math import sqrt
- def dis(x1,y1,x2,y2=5): #求平 面上两点距离
- print("x1={},y1={},x2={},y2={}".format(x1,y1,x2,y2))
- return sqrt((x1-x2)**2+(y1-y2)**2)
- print(dis(1,3,4))

默认参数

- 输入的参数可以是事先设定好赋值，也就是默认值。在调动函数的时候，可以不输入参数，函数内部会直接调用默认参数值。例如默认 z=3

```
def polynomial(x, y, z=3):
    s = 1 + 2*x + y*y + z*y
    return s
#调用函数
polynomial(x=1, y=2)
```

```
13
```

默认参数

- 默认参数的默认值是可以修改的，将上面的 z 值传入设置为 4
- 需要注意的是，默认参数必须放到参数列表的末位

```
# 调用函数，并输入修改的z参数
polynomial(1, 2, z=4)
```

```
15
```

默认参数值在函数对象被创建时计算

- def init(arg, result=[]):
- 　　result.append(arg)
- 　　print(result)
- init('a')
- init('b')

- 程序输出：
- ['a']
- ['a', 'b']

实例理解：

```
def init( arg, result=[]):
    result.append(arg)
    print( result)
init('a')
init
```

```
['a']

<function __main__.init(arg, result=['a'])>
```

```
init('b')
init
```

```
['a', 'b']

<function __main__.init(arg, result=['a', 'b'])>
```

函数定义与调用时，不定长参数的传入

- 函数定义时
- *可以将按位置传递进来的参数“打包”成元组(tuple)类型
  **可以将按关键字传递进来的参数“打包”成字典(dictionary)类型

- 函数调用时
- *可以“解压”待传递到函数中的 元组、列表、集合、字符串等类型，并按位置传递到函数入口参数中
  **可以“解压”待传递到函数中的字典，并按关键字传递到函数入口参数中

不定长参数*args

- 这是为了解决，不确定需要传入参数个数的情况

```
#*args
def print_keywords(x, *args ):
    for i in args:
        print(x + i)
    return

#调用函数
print_keywords(1, 1, 2, 3)
```

```
2
3
4
```

不定长参数*args

*args

*args 传入的时候，如果调用函数使用关键词传入参数时会出错

```
print_keywords(1, y=1, z=2, m=3)   #用关键词参数传入
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)

<ipython-input-41-04aa614002f1> in ()
----> 1 print_keywords(1, y=1, z=2, m=3)


TypeError: print_keywords() got an unexpected keyword argument 'y'
```



不定长参数*args

*args

- 如果输入是一个 list ， 那么可以用 *list 的方式传入

```
a = [1,2,3]
print_keywords(1, *a)
```

```
2
3
4
```

不定长参数*args

函数定义时，*可以将按位置传递进来的参数“打包”成元组(tuple)类型

函数调用时，*可以“解压”待传递到函数中的 元组、列表、集合、字符串等类型，并按位置传递到函数入口参数中

```
def human(gender,height,*args):
    print 'gender:',gender
    print 'height:',height
    print args
human('female',173,[1,3],[2,3])
```

```
gender: female
height: 123
([1, 3], [2, 3])
```

不定长参数*args

- 当函数参数数目不确定时，星号将一组可变数量的位置参数集合成参数值的元组

```
def countnum(a,*b): #计算参数个数
    print(b)
    print(len(b)+1)
countnum(3,7,9)
countnum(5,8,1,6,89)
```

运行结果：

```
(7, 9)      #以元组的形式存放
3
(8, 1, 6, 89)
5
```

print函数参数

- print(*object,sep=" ",end="\n",file=sys.stdout)
- object：输出参数
- sep=" "：输出分割符
- end="\n"：输出函数结束换行
- file=sys.stdout：输出到屏幕

print是接受可变参数的函数

- 对print()这种能接受可变数量参数的函数是非常有用的
- >>>l=[2,7,5]
- >>>print(l)　#参数个数是一个
- [2, 7, 5]
-
- >>>print(*l)　#实参拆包
- #print(2,7,5)
- 2 7 5

- *或**都是加在形参的前面，表示不定长参数，分别用来接收不带变量名的多余参数和带有变量名的多余参数，分别将它们以元组和字典的形式接收进函数。
- 当在实参的前面加上*或**时，就意味着拆包。*表示将序列拆成一个个单独的实参，而**则表示将字典拆成一个个单独的带变量名的实参。

收集参数到字典中---**

- *或**都是加在形参的前面，表示不定长参数，分别用来接收不带变量名的多余参数和带有变量名的多余参数

- def countnum(a,**d): #注意：**d（区别于*d）将后面几个参数收集到了一个变量d中
- print(d)
- print(len(d)+1)
- countnum(3,x1=9,x2=1,x3=6,x4=89)

- 程序输出：
- {'x1': 9, 'x2': 1, 'x3': 6, 'x4': 89}
- 5

常见包中的不定参数

- scikit-learn中的测试集训练集划分函数

- def train_test_split(*array, **options)

```
def train_test_split(*arrays, **options):
    """Split arrays or matrices into random train and test subsets

    Quick utility that wraps input validation and
    ``next(ShuffleSplit().split(X, y))`` and application to input data
    into a single call for splitting (and optionally subsampling) data in a
    oneliner.

    Read more in the :ref:`User Guide <cross_validation>`.

    Parameters
    ----------
    *arrays : sequence of indexables with same length / shape[0]
        Allowed inputs are lists, numpy arrays, scipy-sparse
        matrices or pandas dataframes.

    test_size : float, int, or None (default is None)
        If float, should be between 0.0 and 1.0 and represent the
        proportion of the dataset to include in the test split. If
        int, represents the absolute number of test samples. If None,
        the value is automatically set to the complement of the train size.
        If train size is also None, test size is set to 0.25.

    train_size : float, int, or None (default is None)
        If float, should be between 0.0 and 1.0 and represent the
        proportion of the dataset to include in the train split. If
        int, represents the absolute number of train samples. If None,
        the value is automatically set to the complement of the test size.

    random_state : int or RandomState
        Pseudo-random number generator state used for random sampling.

    stratify : array-like or None (default is None)
        If not None, data is split in a stratified fashion, using this as
        the class labels.
```

当函数的形参值改变后，会影响相应的实参吗？

- 当实参是**不**可变对象时，形参值改变**不**会影响实参！
- 当实参是可变对象时，形参值改变可能会影响实参！

- 可变对象与不可变对象的区别在于对象本身是否可变。
  - 可变对象：list dict set
  - 不可变对象：tuple string int float bool

当函数的形参值改变后，会影响相应的实参吗？

地址问题

下面我们来看一下可变对象的内存地址变化

```
>>> a = [1, 2, 3]
>>> id(a)
2139167175368
>>> a[1] = 4
>>> id(a)
2139167175368
```

我们可以看到，可变对象变化后，地址是没有改变的

如果两个变量同时指向一个地址

1.可变对象

```
>>> a = [1, 2, 3]
>>> id(a)
2139167246856
>>> b = a
>>> id(b)
2139167246856
>>> a[1] = 4
>>> a
[1, 4, 3]
>>> b
[1, 4, 3]
>>> id(a)
2139167246856
>>> id(b)
2139167246856
```

我们可以看到，改变a则b也跟着变，因为他们始终指向同一个地址

2.不可变对象

```
>>> a = (1, 2, 3)
>>> id(a)
2139167074776
>>> b = a
>>> a = (4, 5, 6)
>>> a
(4, 5, 6)
>>> b
(1, 2, 3)
>>> id(a)
2139167075928
>>> id(b)
2139167074776
```

我们可以看到，a改变后，它的地址也发生了变化，而b则维持原来的地址，原来地址中的内容也没有发生变化

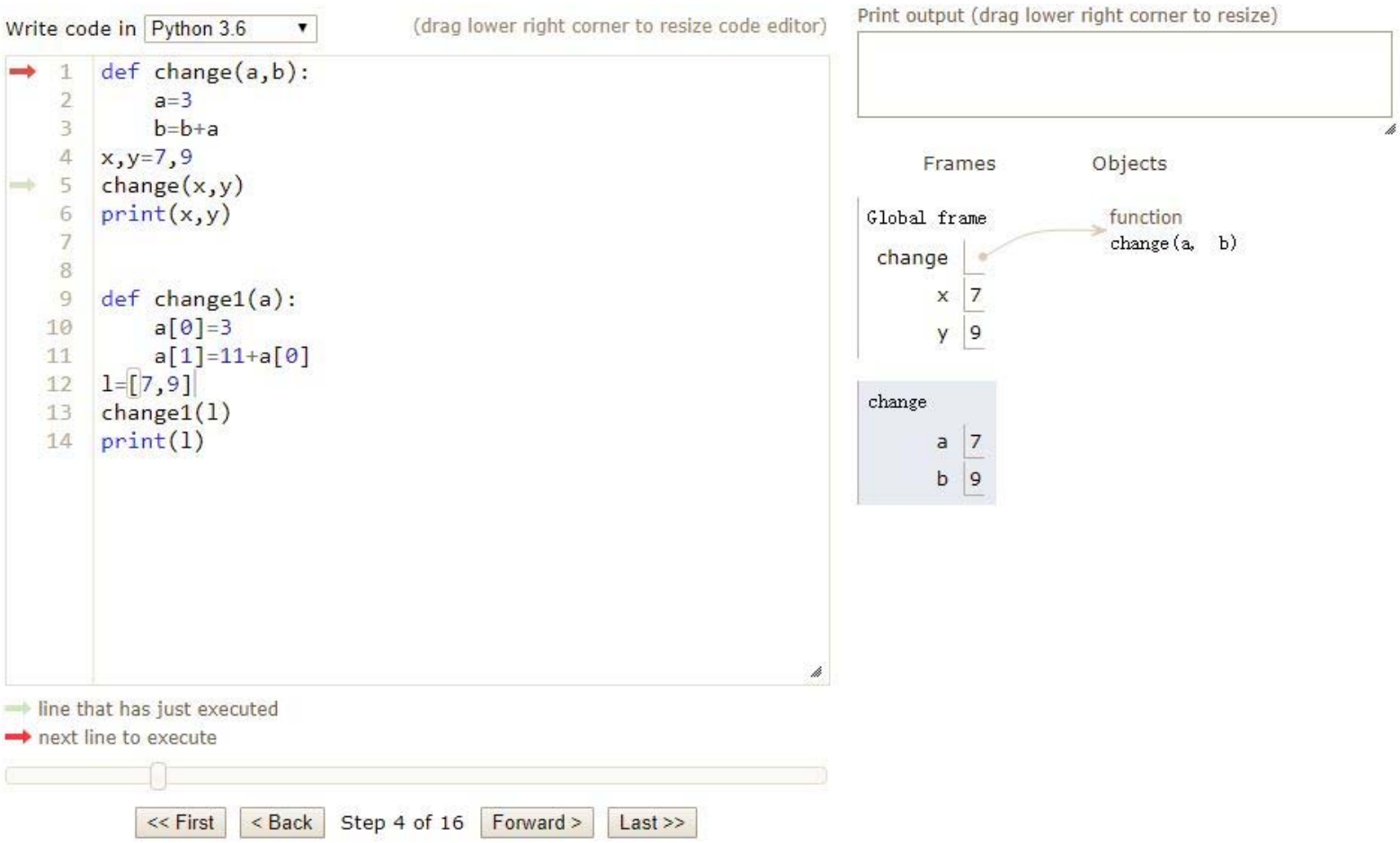
实参和形参（1）

实例理解：

```
def change(a,b):
    a=3
    b=b+a
x,y=7,9
change(x,y)
print(x,y)


def change1(a):
    a[0]=3
    a[1]=11+a[0]
1=[7,9]
change1(1)
print(1)
```

```
7 9
[3, 14]
```



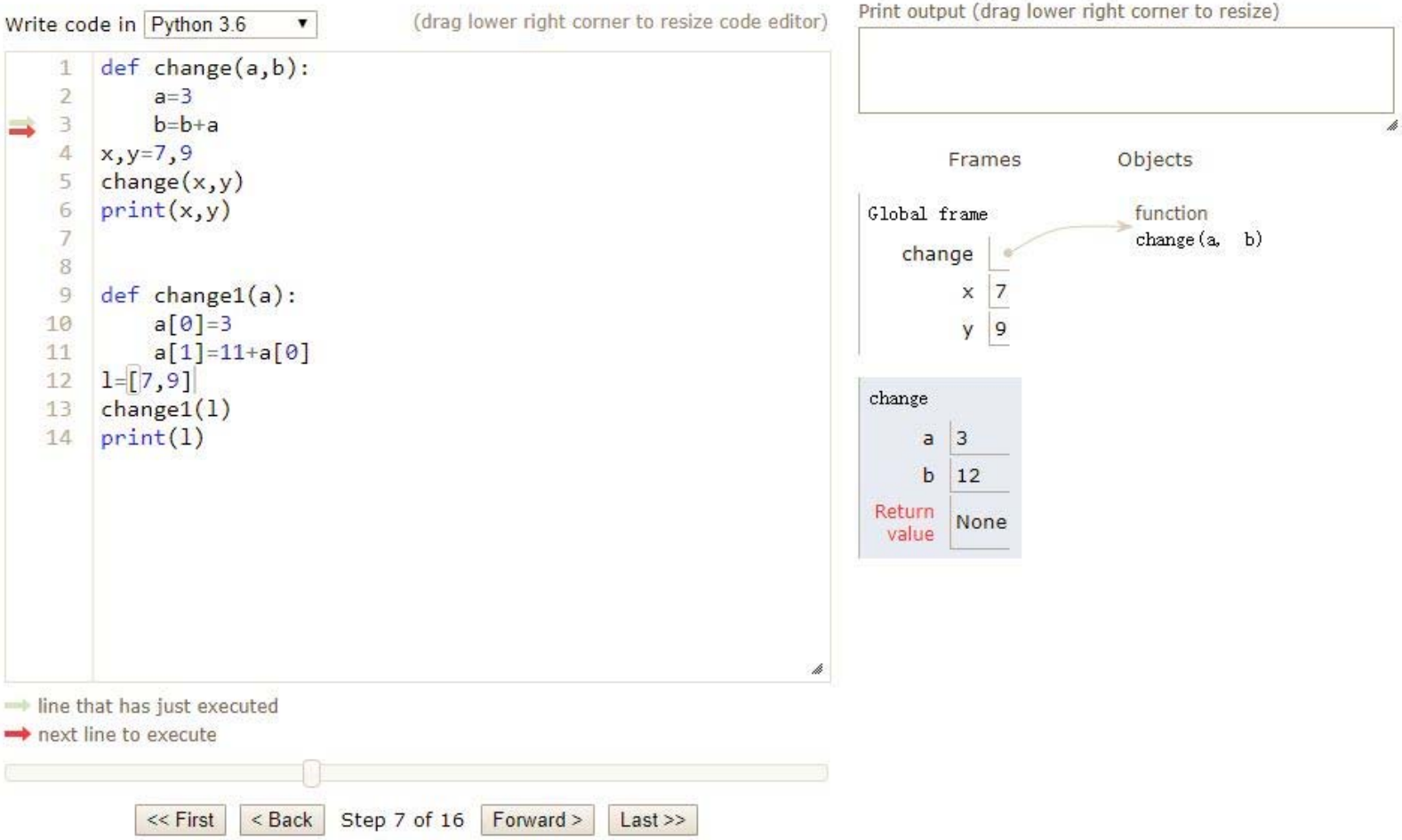
实参和形参（2）

实例理解：

```
def change(a,b):
    a=3
    b=b+a
x,y=7,9
change(x,y)
print(x,y)


def change1(a):
    a[0]=3
    a[1]=11+a[0]
l=[7,9]
change1(l)
print(l)
```

```
7 9
[3, 14]
```



实参和形参（3）

实例理解：

```
def change(a,b):
    a=3
    b=b+a
x,y=7,9
change(x,y)
print(x,y)


def change1(a):
    a[0]=3
    a[1]=11+a[0]
1=[7,9]
change1(1)
print(1)
```

```
7 9
[3, 14]
```

Write code in Python 3.6 (drag lower right corner to resize code editor)

```
1  def change(a,b):
2      a=3
3      b=b+a
4  x,y=7,9
5  change(x,y)
6  print(x,y)
7
8
9  def change1(a):
10     a[0]=3
11     a[1]=11+a[0]
12 1=[7,9]
13 change1(1)
14 print(1)
```

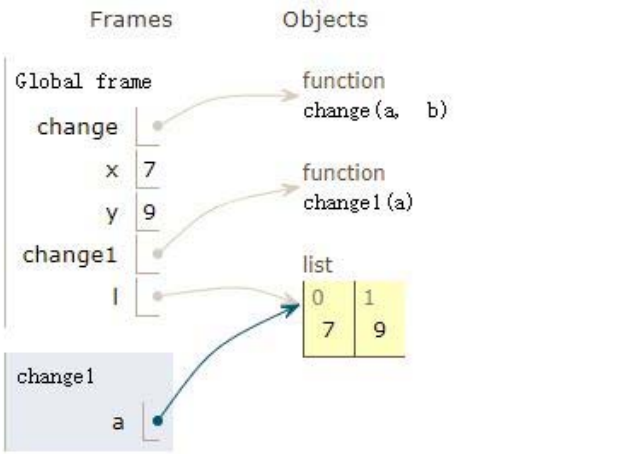
line that has just executed

next line to execute

<< First | < Back | Step 12 of 16 | Forward > | Last >>

Print output (drag lower right corner to resize)

```
7 9
```



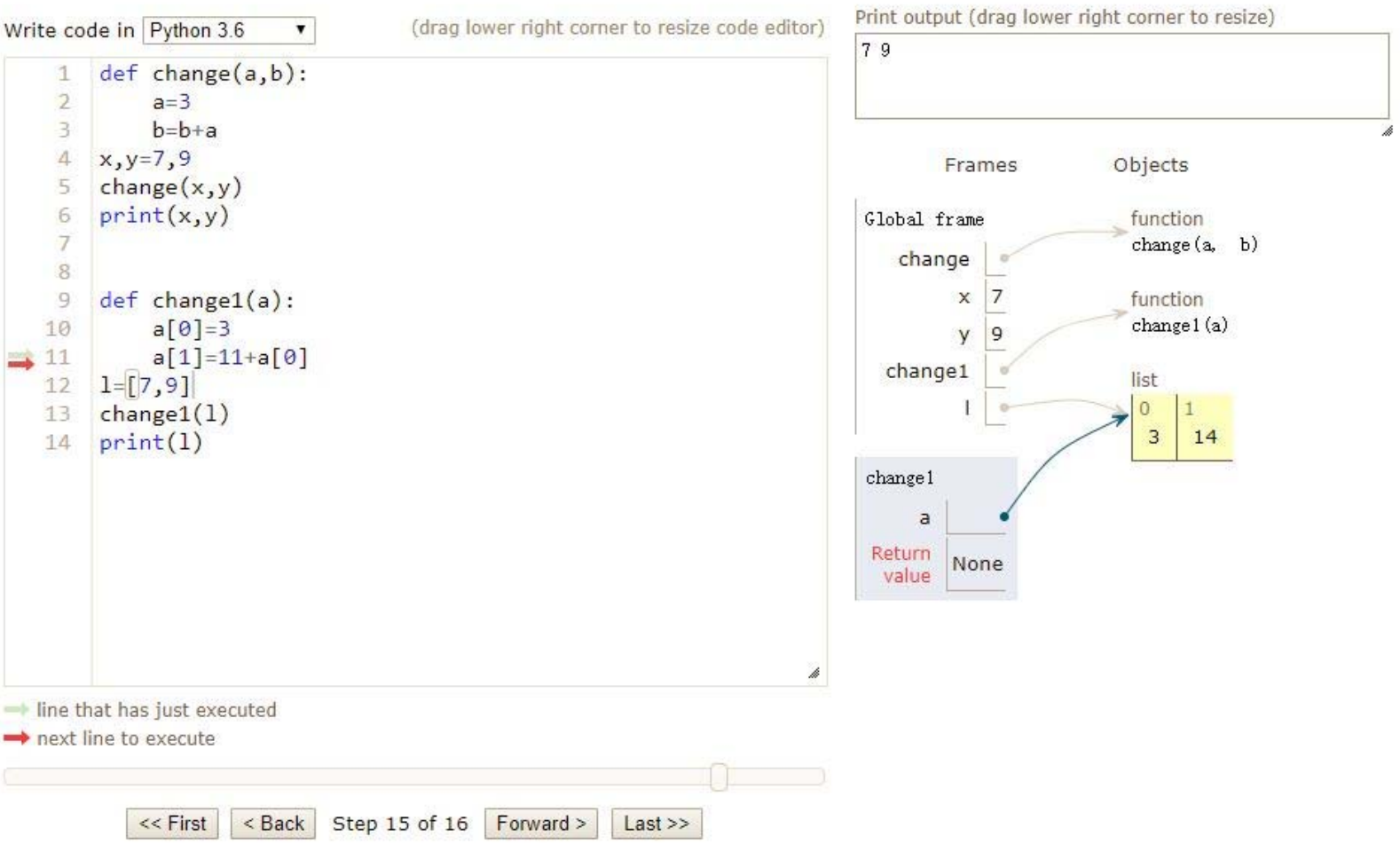
实参和形参（4）

实例理解：

```
def change(a,b):
    a=3
    b=b+a
x,y=7,9
change(x,y)
print(x,y)


def change1(a):
    a[0]=3
    a[1]=11+a[0]
l=[7,9]
change1(l)
print(l)
```

```
7 9
[3, 14]
```



函数的返回值

- 函数用return语句返回值。
- return后面的表达式的值就成为这次函数调用的返回值。
- 如函数没有用return语句返回，这时函数返回的值为None； 如果return后面没有表达式，调用的返回值也为None。
- None是Python中一个特殊的值，虽然它不表示任何数据，但仍然具有重要的作用。

整数M和N区间内素数的个数并对它们求和(2<=M<N)

```
def isprime(i):
    for k in range(2,i):
        if i%k==0:
            return False
    return True

m,n=input().split()
m,n=int(m),int(n)
p=[i for i in range(m,n+1) if isprime(i)]
print(len(p),sum(p))
```

函数和字典的组合

```
def test(par):
    print(par)

def test1():
    print(1000)

def test3(par):
    print(2000)

def f(x):
    return {
        'a': test,
        'b': test1,
        }.get(x, test3)

print(f('a')(100))
print(f(4)(100))
```

去掉列表中的重复元素并按原顺序输出

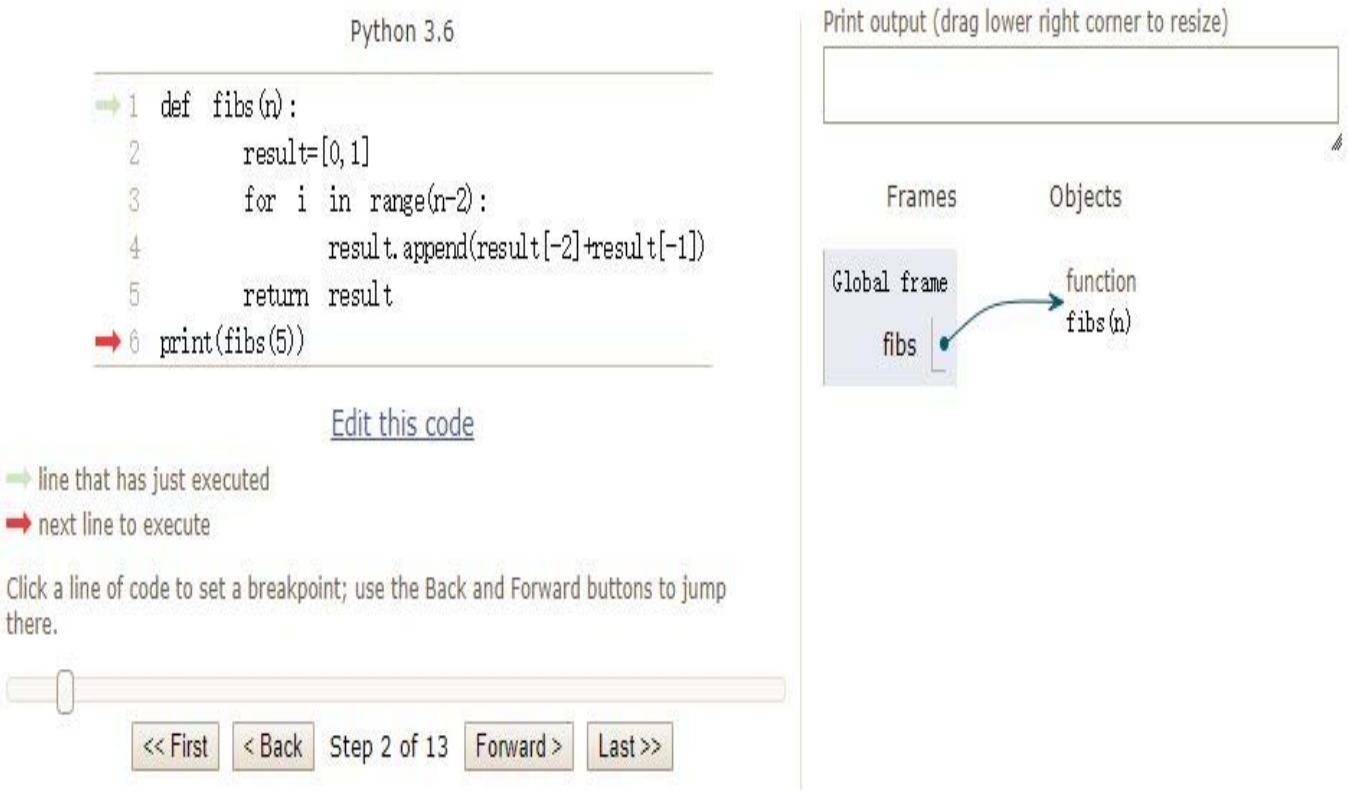
- l=[2,3,5,8,3,6,8]
- seen=set()
- l1=[i for i in l if i not in seen and not seen.add(i)]
- print(l1)

- 下面程序对吗?
- l=[2,3,5,8,3,6,8]
- seen=set()
- l1=[i for i in l if not seen.add(i) and i not in seen]
- print(l1)

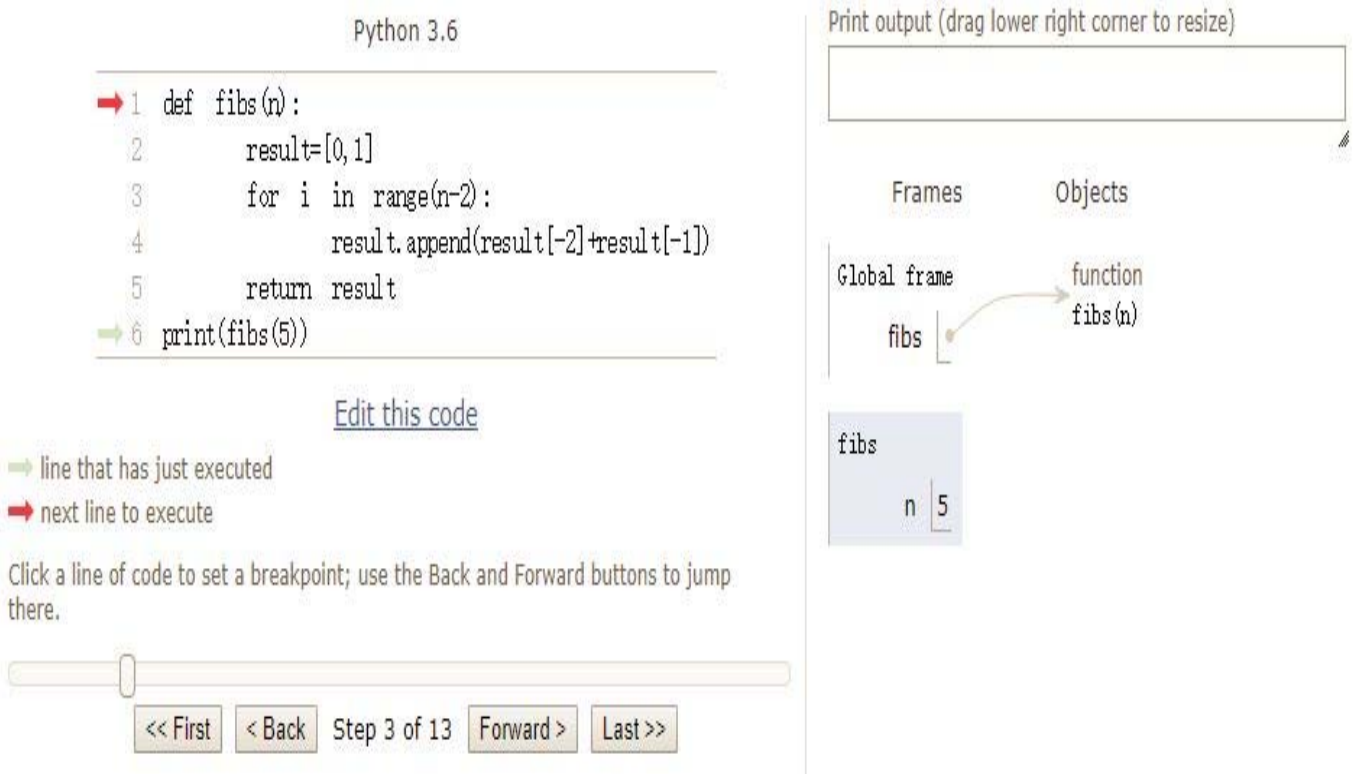
例 求f(n)的前n项值

```
def fibs(n):                    函数定义
    result=[0,1]
    for i in range(n-2):
        result.append(result[-2]+result[-1])
    return result  #返回列表，c语言返回?
print(fibs(5))      函数调用
```

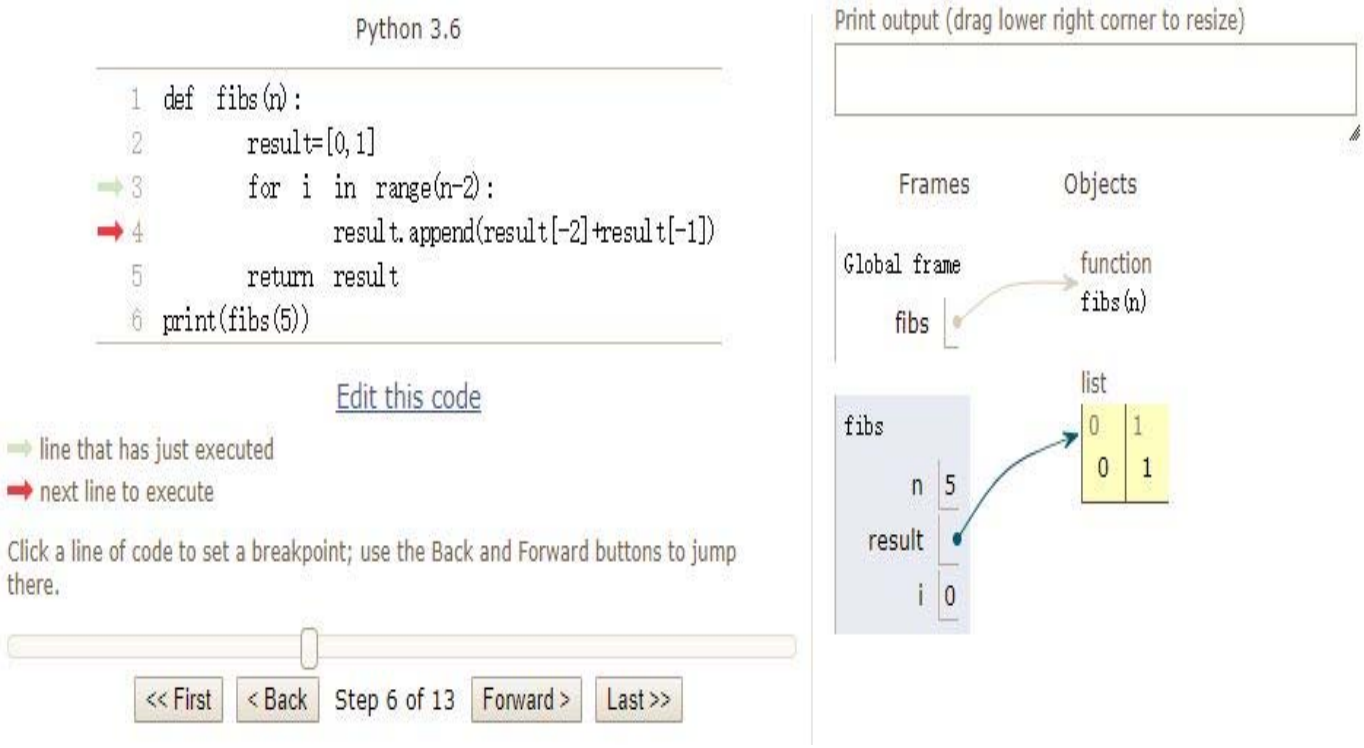
运行过程---创建fibs函数对象



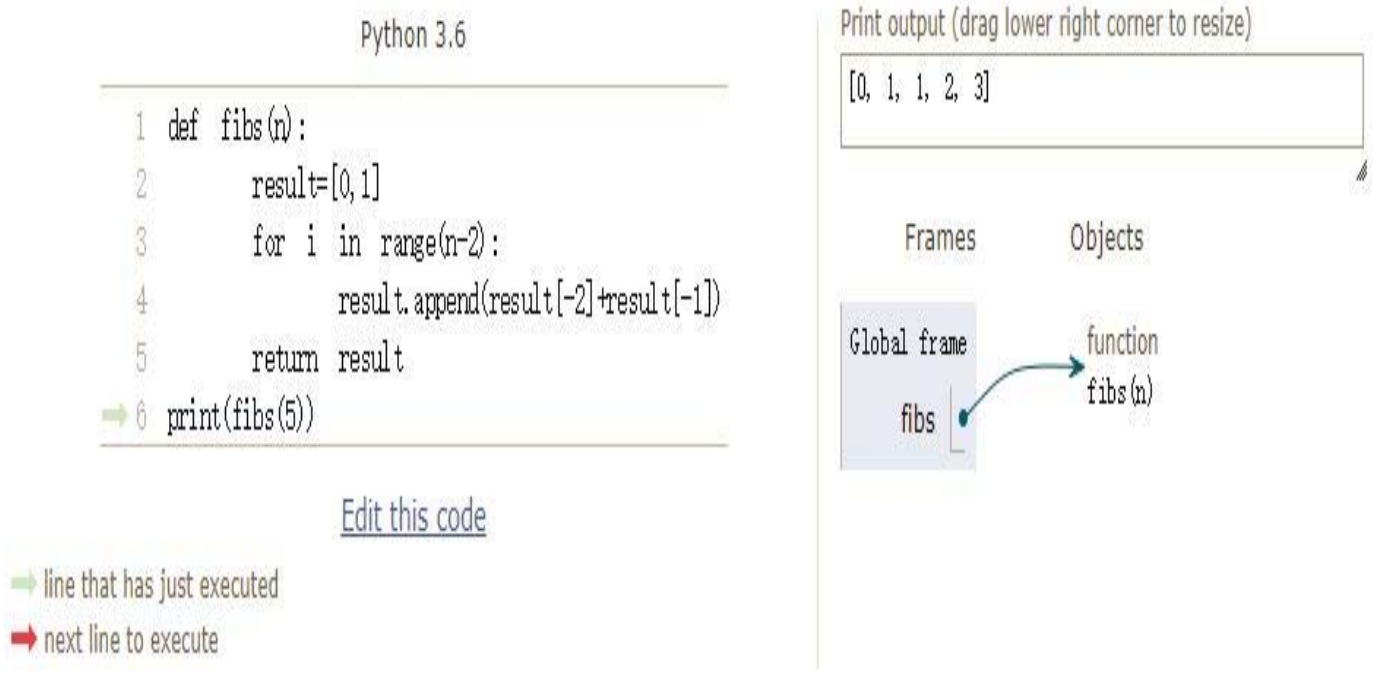
参数传递


Python 3.6
1 def fibs(n):
2 result=[0,1]
3 for i in range(n-2):
4 result.append(result[-2]+result[-1])
5 return result
6 print(fibs(5))
Edit this code
line that has just executed
next line to execute
Click a line of code to set a breakpoint; use the Back and Forward buttons to jump there.
<< First
< Back
Step 3 of 13
Forward >
Last >>
Print output (drag lower right corner to resize)
Frames
Objects
Global frame
fibs
function
fibs(n)
fibs
n
5

执行结果存在列表中


Python 3.6
1 def fibs(n):
2 result=[0,1]
3 for i in range(n-2):
4 result.append(result[-2]+result[-1])
5 return result
6 print(fibs(5))
Edit this code
line that has just executed
next line to execute
Click a line of code to set a breakpoint; use the Back and Forward buttons to jump there.
<< First
< Back
Step 6 of 13
Forward >
Last >>
Print output (drag lower right corner to resize)
Frames
Objects
Global frame
fibs
function
fibs(n)
fibs
n 5
result
i 0
list
0 1
0 1

函数调用结束返回结果


Python 3.6
1 def fibs(n):
2 result=[0,1]
3 for i in range(n-2):
4 result.append(result[-2]+result[-1])
5 return result
6 print(fibs(5))
Edit this code
line that has just executed
next line to execute
Print output (drag lower right corner to resize)
[0, 1, 1, 2, 3]
Frames
Objects
Global frame
fibs
function
fibs(n)

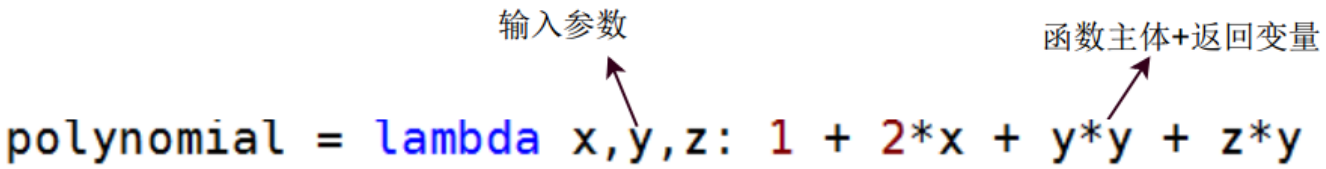
lambda函数

函数定义的另一种方法是用lambda表达式，它定义了一个匿名函数。lambda的一般形式是关键字lambda后面跟一个或多个参数，紧跟一个冒号，后面是一个表达式。作为表达式，lambda返回一个值。lambda 用来编写简单的函数，而def用来处理更强大的任务的函数。

```
g = lambda x,y,z : x+y+z
#把lambda定义的匿名函数赋给函数g
print（g(1,4,5)）
```

lambda函数

- 简化的函数定义方式




```
polynomial = lambda x,y,z:1 + 2*x + y*y + z*y

polynomial(1, 2, 3)
```

```
13
```

函数要先定义，再使用

- print(f(3))

- def f(x):
- 	return x**2+1

- def f(x):
- 	return x**2+1

- print(f(3))

- 错

递归

函数调用自身的编程技巧称为递归

递归做为一种算法在程序设计语言中广泛应用。它通常把一个大型复杂的问题层层转化为一个与原问题相似的规模较小的问题来求解，大大地减少了程序的代码量。

递归的能力在于用有限的语句来定义对象的无限集合。一般来说，递归需要终止条件和递归条件。当终止条件不满足时，递归前进；当终止条件满足时，递归返回。编写递归函数时，必须告诉它何时停止递归，直接返回结果，从而避免形成无限循环。

斐波那契数列

- 斐波那契数列是一个经常使用递归方式定义的常用数学函数.
- fib(0)= 1　　n=0　　(1)
- fib(1)= 1　　n=1　　(2)
- fib(n)=fib(n-1)+fib(n-2)　n>=2　　(3)
- 公式（1）和公式（2）是终止条件，公式（3）是递归条件

斐波那契数列递归程序

- def fib(n): #假定n是正整数返回第n个斐波那契数
- 	if n == 0 or n == 1:
- 		return 1
- 	else:
- 		return fib(n-1) + fib(n-2)
- print(fib(4))
- 思考：fib(100)要算多久?

改进斐波那契数列递归程序

```
pre={0:1,1:1}
def fib(n):
    if n in pre:  #可以用in检查字典中是否有n这个关键字
        return pre[n]
    else:
        newvalue=fib(n-1)+fib(n-2)
        pre[n]=newvalue #增加字典的条目
        return newvalue
print(fib(100))
```

求嵌套列表中数字元素和

```
def flatten(items):
    lst=[]
    for x in items:
        if isinstance(x,(list,tuple)) and not isinstance(x,str):
            for element in flatten(x):
                lst.append(element)
        else:
            if type(x)!=str:
                lst.append(x)
    return lst

#items=[11,2,[3,7],(68,-1),"123",9]
items=eval(input()) #列表输入方法
l=[i for i in flatten(items)]
print(sum(l))
```

内置函数

		Built-in Functions		
abs()	dict()	help()	min()	setattr()
all()	dir()	hex()	next()	slice()
any()	divmod()	id()	object()	sorted()
ascii()	enumerate()	input()	oct()	staticmethod()
bin()	eval()	int()	open()	str()
bool()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	
delattr()	hash()	memoryview()	set()	

sorted函数

- sorted函数对字符串，列表，元组，字典等对象进行排序操作。
- sort是应用在list上的方法，sorted可以对更多的数据类型进行排序操作。
- 即便都是对列表操作，list的sort方法返回的是对已经存在的列表进行操作，而内建函数sorted返回的是一个新的list，而不是在原来的基础上进行的操作。
-

sorted函数语法

- sorted(iterable[,key[, reverse]])
- iterable -- 序列，如字符串，列表，元组等。
- key -- 主要是用来进行比较的元素，只有一个参数，具体的函数的参数就是取自于可迭代对象中，指定可迭代对象中的一个元素来进行排序。
- reverse -- 排序规则
- reverse = True 降序， reverse = False 升序（默认）。
-

表格排序

姓名	分数	年龄
江幸	89	15
方鹏	80	14
陈可	85	14

- >>>students = [('江幸',89, 15), (' 方鹏',80, 14), ('陈可', 85, 14)]
- #第二个分量是成绩，第三个分量是年龄
-
- >>>print(sorted(students, key=lambda s: s[2])) # 按年龄从小到大排序
- [(' 方鹏', 80, 14), ('陈可', 85, 14), ('江幸', 89, 15)]
-
-
- >>>print(sorted(students, key=lambda s: s[1], reverse=True))# 按成绩从大到小降序
- [('江幸', 89, 15), ('陈可', 85, 14), (' 方鹏', 80, 14)]

map函数

- map会根据提供的函数对指定序列做映射。
- map函数语法：map(function, iterable, ...)
- 第一个参数 function是以参数序列中的每一个元素调用 function 函数，iterable是序列，返回包含每次 function 函数返回值的新列表或迭代器 。

map函数举例

- >>>print(list(map(lambda x: x ** 2, [1, 2, 3, 4, 5]))) # 使用 lambda 匿名函数
- [1, 4, 9, 16, 25]
-
-
- >>>print(list(map(lambda x, y: x + y, [1, 3, 5, 7, 9], [2, 4, 6, 8, 10])))
- # 提供了两个列表，对相同位置的列表数据进行相加
- [3, 7, 11, 15, 19]

zip函数

- zip() 函数用于将可迭代的对象作为参数，将对象中对应的元素打包成一个个元组，然后返回由这些元组组成的列表或迭代器 。
- 如果各个迭代器的元素个数不一致，则返回列表长度与最短的对象相同。
- zip 语法：zip([iterable, ...])

  参数说明：iterable -- 一个或多个序列

  返回值：返回元组列表

zip函数举例

- >>>a = [1,2,3]
- >>>b = [4,5,6]
- >>>c = [4,5,6,7,8]

- print(list(zip(a,b)))
- [(1, 4), (2, 5), (3, 6)]
- print(list(zip(a,c))) # 元素个数与最短的列表一致
- [(1, 4), (2, 5), (3, 6)]

字典键值互换

- >>>d={'blue':500,'red':100,'white':300}
- >>>d1=dict(zip(d.values(),d.keys())
- >>>print(d1)
- {500: 'blue', 100: 'red', 300: 'white'}

eval和exec函数

- Python是一种动态语言，它包含很多含义。Python变量类型，操作的合法性检查都在动态运行中检查；运算的代码需要到运行时才能动态确定；程序结构也可以动态变化，容许动态加载新模块等。这两个函数就体现了这个特点。
- eval是计算表达式,返回表达式的值。
- exec可运行Python的程序,返回程序运行结果。

eval和exec函数举例

- >>>x,y=3,7
- >>>eval('x+3*y-4')
- 20

- >>>exec('print("hello world")')
- Hello world

all和any函数

- all()和any()函数将可迭代的对象作为参数。
- all函数参数都是True时，才返回True，否则返回False
- any函数参数只要有一个为True，就返回True,参数全部是False时，返回False.

all和any函数举例

- >>> n=47
- >>> all([1 if n%k!=0 else 0 for k in range(2,n)])
- True
- 思考：True背后的含义？
- >>> n=15
- >>> all([1 if n%k!=0 else 0 for k in range(2,n)])
- False
- >>> any([[],False,0])　#空列表和0都表示False
- False

程序结构

- 书的内容按照这样的层次组织：单词、句子、段落以及章。
- 代码也有类似的自底向上的组织层次：数据类型类似于单词，语句类似于句子，函数类似于段落，模块类似于章。
- 一个Python程序可以在不同文件中，这些文件是一个个模块。用import语句引入。
- Import 模块名
- 模块名是另外一个Python文件的文件名,不包含扩展名

引入模块中的函数另一方法

from 模块名 import *

这种方法引入模块中的所有函数，调用的时候不需要再加模块名

from 模块名 import 函数名

这种方法引入模块中的单个函数，调用的时候也不需要再加模块名

创建模块(两文件同一目录)

下面是triangle模块（文件名为 triangle.py）

```
import math
def area(a,b,c):
    s=(a+b+c)/2
    return (math.sqrt(s*(s-a)*(s-b)*(s-c)))
```

主程序是另一个文件（area.py）

```
import triangle
a=12
b=34
c=26
print(triangle.area(a,b,c))
```

模块名字空间

- 1.包含“主程序”（启动程序的代码）的模块
  的模块空间就是全局名字空间
- 2.模块空间有:
- 模块名是模块的文件名, 包含“主程序”的
  模块名是”__main__”
  全局变量
  函数名

3.dir(模块名）：显示模块属性

包

- 我们已使用过单行代码、多行函数、独立程序以及同一目录下的多个模块。为了使Python应用更具可扩展性，你可以把多个模块文件组织成目录，称之为包。包是一个目录，其中包含一组模块文件和一个*init.py*文件。如果模块存在于包中，使用“import 包名.模块名”形式导入包中模块，用以下形式调用函数：“包名.模块名.函数”

sys模块

如果area.py和triangle.py上述两个文件在同一个目录下，通过Python运行主程序area.py,会引用triangle模块，执行函数area。

模块的查找路径：sys.path

如不在同一目录，可用sys模块加入搜索路径后调用。

sys模块中的常用函数

函数名	函数功能
sys.argv	从程序外部向程序传递参数
sys.exit([arg])	程序中间的退出，arg=0为正常退出
sys.getdefaultencoding()	获取系统当前编码。
sys.setdefaultencoding()	设置系统默认编码
sys.getfilesystemencoding()	获取文件系统使用的编码方式，Windows下返回'mbcs'，mac下返回'utf-8'
sys.path	模块搜索路径的字符串列表
sys.platform	获取当前系统平台
sys.stdin sys.stdout sys.stderr	stdin , stdout , 以及stderr 变量包含与标准I/O 流对应的流对象. 如果需要更好地控制输出,而print不能满足你的要求, 它们就是你所需要的. 你也可以替换它们, 这时候你就可以重定向输出和输入到其它设备(device), 或者以非标准的方式处理它们

命令行参数

- 如何不用input()函数向程序输入值？命令行参数可以解决这个问题
- sys.argv[0]　#程序的文件名
- sys.argv[1]　第一个参数
- sys.argv[2]　第二个参数
- ……
- sys.argv[n]　第n个参数

命令行参数例子

- import sys
- print(sys.argv[0])　#程序的文件名
- print(sys.argv[1])　#被乘数
- print(sys.argv[2])　#乘数
- print(int(sys.argv[1])*int(sys.argv[2]))
- 程序执行：
- c:\>python d:\mul.py 2 6
- d:\mul.py
- 2
- 6
- 12

总结

- 函数的语法规则
- 位置传入、关键词、默认参数、不定长参数
  参数定义的顺序：必选参数、默认参数、可变参数和关键字参数
- lambda函数

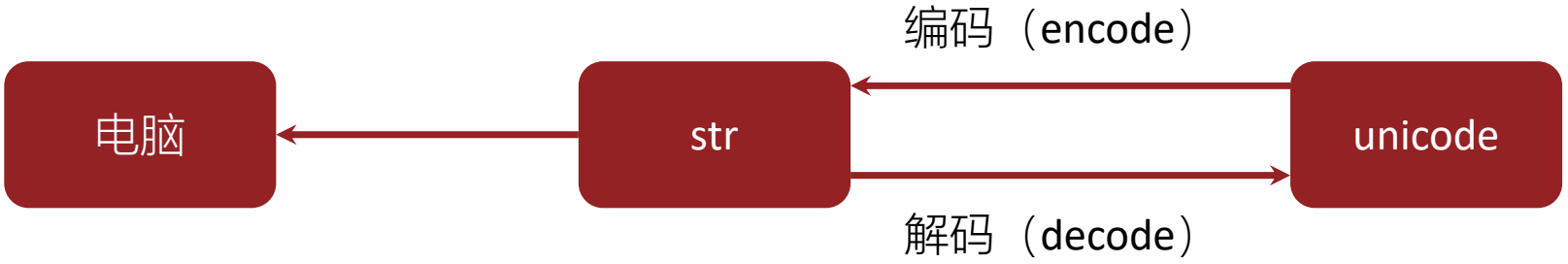


第八部分 字符编码问题处理

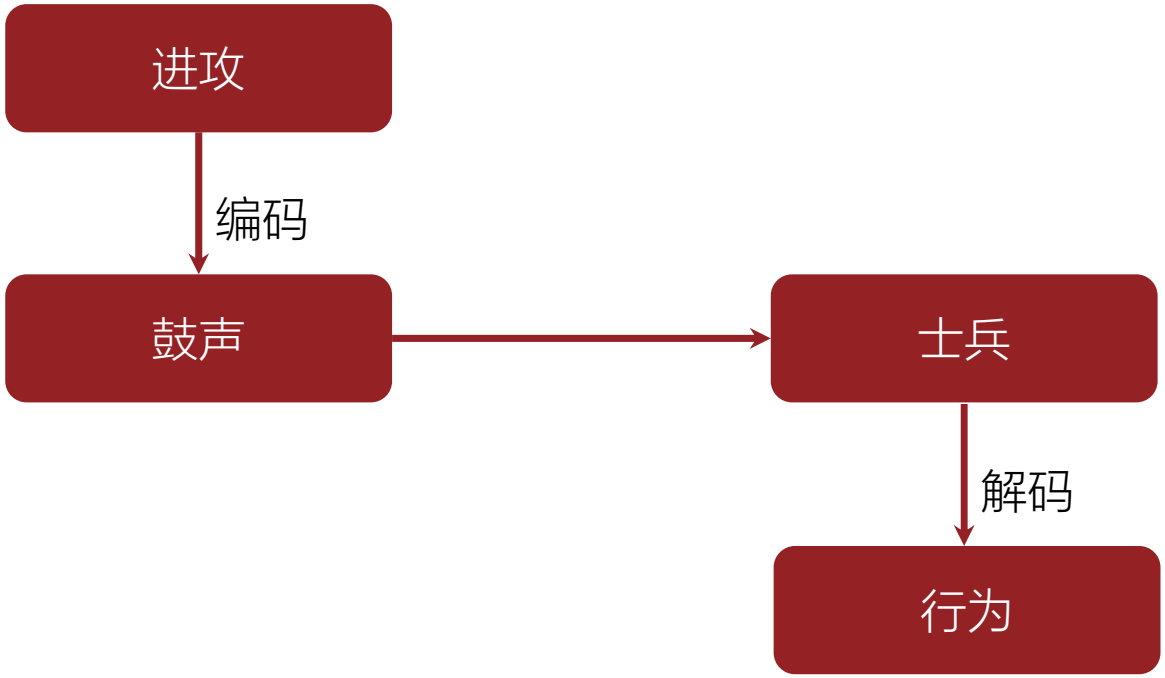
编码和解码
编码集
解决乱码

编码和解码

- 编码是为了让机器读懂语言。即输入的是字符“中文”，那么编码就是将“中文”编码为二进制格式让机器读懂
- 在Python中，机器其实是不认识 unicode 的，而是接收的 str
- 即使输入的数据是 unicode 的，而在Python内部都会将 unicode 自动转为 str



编码和解码



编码集

- 电脑只"认识" 0 1 二进制数值，从一开始，人们想到的方法就是赋予每一个字符唯一的一串 0 1 编码表示。

0	1	0	0	0	0	0	1

- 这是一个ASCII编码集中对字符 A 的编码，这是一个八位 0/1 数字

ASCII编码集

- 在1967年出现了ASCII标准，这主要是基于拉丁字母的一套电脑编码系统
- ASCII标准是每个英文字母有一一对应独立的 8 位数字串，比如说字母 A，ASCII中对应的二进制值为 01000001 。这样，我在输入 A 的时候，编译器会借助编码集将 A 转换为 01000001

0	1	0	0	0	0	0	1

ASCII编码集

- ASCII总共可以存储256个字符，1967年制定这套标准的人只考虑英语世界的使用者。那么256个编码集对于使用英语和拉丁语系是够用的，但是对于中文、日文、韩文，就完全不够用了

250	372	FA	11111010	ú	ú	ú	Latin small letter u with acute
251	373	FB	11111011	û	û	û	Latin small letter u with circumflex
252	374	FC	11111100	ü	ü	ü	Latin small letter u with diaeresis
253	375	FD	11111101	ý	ý	ý	Latin small letter y with acute
254	376	FE	11111110	þ	þ	þ	Latin small letter thorn
255	377	FF	11111111	ÿ	ÿ	ÿ	Latin small letter y with diaeresis

ASCII编码集

DEC	OCT	HEX	BIN	Symbol	HTML Number	HTML Name	Description
32	040	20	00100000				Space
33	041	21	00100001	!	!		Exclamation mark
34	042	22	00100010	"	"	"	Double quotes (or speech marks)
35	043	23	00100011	#	#		Number
36	044	24	00100100	$	$		Dollar
37	045	25	00100101	%	%		Procenttecken
38	046	26	00100110	&	&	&	Ampersand
39	047	27	00100111	'	'		Single quote
40	050	28	00101000	(	(		Open parenthesis (or open bracket)
41	051	29	00101001	)	)		Close parenthesis (or close bracket)
42	052	2A	00101010	*	*		Asterisk
43	053	2B	00101011	+	+		Plus
44	054	2C	00101100	,	,		Comma
45	055	2D	00101101	-	-		Hyphen
46	056	2E	00101110	.	.		Period, dot or full stop
47	057	2F	00101111	/	/		Slash or divide
48	060	30	00110000	0	0		Zero
49	061	31	00110001	1	1		One
50	062	32	00110010	2	2		Two
51	063	33	00110011	3	3		Three
52	064	34	00110100	4	4		Four
53	065	35	00110101	5	5		Five
54	066	36	00110110	6	6		Six
55	067	37	00110111	7	7		Seven
56	070	38	00111000	8	8		Eight
57	071	39	00111001	9	9		Nine
58	072	3A	00111010	:	:		Colon
59	073	3B	00111011	;	;		Semicolon
60	074	3C	00111100	<	<	<	Less than (or open angled bracket)
61	075	3D	00111101	=	=		Equals
62	076	3E	00111110	>	>	>	Greater than (or close angled bracket)
63	077	3F	00111111	?	?		Question mark
64	100	40	01000000	@	@		At symbol
65	101	41	01000001	A	A		Uppercase A
66	102	42	01000010	B	B		Uppercase B
67	103	43	01000011	C	C		Uppercase C
68	104	44	01000100	D	D		Uppercase D
69	105	45	01000101	E	E		Uppercase E
70	106	46	01000110	F	F		Uppercase F
71	107	47	01000111	G	G		Uppercase G

GBK

- GB2312编码集是由中国国家标准总局发布
- 1981年5月1日实施的一套收录了6763个汉字、拉丁字母、希腊字母、日文平假名及片假名字母、俄语西里尔字母在内的682个字符的标准。但是这套标准没有包含部分罕见字和繁体字，于是后面出现GBK、GB18030在GB2312的基础上进行扩充解决。

B3	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
A		场	尝	常	长	偿	肠	厂	敞	畅	唱	倡	超	抄	钞	朝
B	嘲	潮	巢	吵	炒	车	扯	撤	掣	彻	澈	郴	臣	辰	尘	晨
C	忱	沉	陈	趁	衬	撑	称	城	橙	成	呈	乘	程	惩	澄	诚
D	承	逞	骋	秤	吃	痴	持	匙	池	迟	弛	驰	耻	齿	侈	尺
E	赤	翅	斥	炽	充	冲	虫	崇	宠	抽	酬	畴	踌	稠	愁	筹
F	仇	绸	瞅	丑	臭	初	出	橱	厨	躇	锄	雏	滁	除	楚	

Unicode

- Unicode是标准统一所有语言文字的标准编码集，它目前已经收录超过十万个字符
- Unicode包括两个方面
  - 编码方式（一个字符对应编码集中的二进制数值）
  - 实现方式（传输、存储）：UTF-8、UTF-16



Python 3.x版本中的字符串

- Python 3.x版本有两种表示字符串的类型： bytes 和 str， str 类中包含了 unicode 字符，非 unicode 编码如 UTF-8、GBK被定义成了 bytes
- 使用 encode() 和 decode() 函数进行相互转换

```
# 本段代码将在Python 3.x版本中运行

c = '中文'
print (type(c),c)

c = c.encode('utf-8') #使用UTF-8将 str 类型编码为 bytes 类型
print (type(c),c)

c = c.decode() #使用默认方式将 bytes 类型解码为 str 类型
print (type(c),c)
```

```
< class 'str'> 中文
< class 'bytes'> b'\xe4\xb8\xad\xe6\x96\x87'
< class 'str'> 中文
```

解决乱码

- 如果出现乱码的状况，那肯定是编码转码出现了问题，这不外乎有以下几个关键点：

  - Python解释器设置的默认编码
  - 源文件设置的编码
  - 终端编码

解释器编码

- Python解释器的编码指Python内部认为的字符 str 的编码，即一个 str 变量，要转换为 unicode 时，Python解释器默认的转换方式
- Python的默认转换方式是utf-8， sys.getdefaultencoding() 函数可以获得默认编码信息

```
>>> import sys
>>> print(sys.getdefaultencoding())
utf-8
>>>
```

解释器编码

- 通过前面介绍的 encode() 函数和 decode() 函数可以设置参数，指定编码方式
- 对中文字符进行 GBK 编码和 UTF-8 编码

```
a = u'中文'
a.encode('gbk')
```

```
'\xd6\xd0\xce\xc4'
```

```
a = u'中文'
a.encode('utf-8')
```

```
'\xe4\xb8\xad\xe6\x96\x87'
```

解释器编码

- 中文字符 str 的解码问题

```
a = '中文'
a.encode('utf-8')
```

```
b'\xe4\xb8\xad\xe6\x96\x87'
```

```
a.encode('utf-8').decode('utf-8')
```

```
---------------------------------------------------------------------------
UnicodeDecodeError                        Traceback (most recent call last)
<ipython-input-42-11e88e4dd477> in <module>()
      1 a = '中文'
----> 2 a.encode('utf-8').decode('gbk')

UnicodeDecodeError: 'gbk' codec can't decode byte 0xad in position 2: illegal multibyte sequence
```

源文件编码

- 源文件编码也就是代码文件的编码，文件不仅可以保存代码，而且还可以对数据进行存储
- 文件保存数据也是有编码的，如果源文件中有中文，那么直接按照默认的编码方式运行Python文件的时候会出现编码错误，若想要在源码中写非ASCII文字，可以在源码文件开头加入一行：

```
# coding: utf-8
```

- 或者

```
# coding: gbk
```



终端编码

- 这个指使用终端显示的时候会出现的问题
- windows中文版终端用的是CP936，通常被等同认为是GBK
- 而linux系统终端用的是utf-8
- 所以若不想在终端打印显示出来的字符出现乱码，则在打印前需要将打印的内容按照终端相同的编码方式进行转换

文件读写编码

- 在数据分析中，出现编码问题最多的还是在文件读写这个阶段
- 使用 read() 读取的文件内容，得到的是 str 格式的

```
f = open( 'train_x.txt' , 'r')
content = f.readlines( )#数据是一个嵌套的列表的格式f.close( )
print(type( content[1][0]))
```

```
'str'
```

```
print (content[1][0]) #直接打印
```

随机数可以用于数学，游戏，安全等领域中，还经常被嵌入到算法中，用以提高算法效率，并提高程序的安全性。

编码检测工具——chardet

- 如果你发现常用的 gbk 、 utf-8 编码解码都没办法解决乱码问题，那么有可能该文本使用了这两种之外的编码方式
- chardet是一个Python包，提供测试文本的编码方法，最后返回的是对编码方法的一个估计

```
import chardet
txt ="随机数".encode('utf-8')
print( chardet.detect(txt))
```

输出：{'encoding': 'utf-8', 'confidence': 0.87625, 'language': ''}

总结

- 编码和解码
- ASCII，GBK，Unicode
- Python 2.x和Python 3.x版本的字符串
- 乱码问题：Python解释器设置的默认编码、源文件的编码、终端编码
- 编码检测工具chardet